

Intrinsic Value Analyzer

Quantitative Stock Market Asset Analysis Model

Model Card V3.0

2026-09-04

Nilrudra Mukhopadhyay

Document Purpose

The following document serves as a report on the Intrinsic Value Analyzer model, showcasing its structure, intended uses, performance in real world tests, and possible future improvements.

Model Background

The intrinsic value analyzer is built on the idea of estimating a stock's "real" price based on a formula introduced by Benjamin Graham, which expresses the Intrinsic Value as a dollar amount of the theoretical "market corrected" share price.

$$V = \frac{EPS \times (8.5 + 2g) \times 4.4}{Y}$$

Figure 1: Benjamin Graham's Intrinsic Value formula

Issues with Graham's formula arise when faced with cyclical businesses where the cash flow is irregular and the P/E changes with seasonal business downturns. The formula inherently requires stocks with continuous stable income and "ideal" circumstances, and while trained analysts can recognize value traps when using this formula, it isn't usable for average investors. Hence, a Machine Learning model can be used to potentially bridge that skill gap through statistically built "intuition".

Model Architecture

The model is broken into two layers, where layer 1 computes an Intrinsic Value as an engineered feature, and layer 2 performs value standardization and contains the Supervised Learning Model, MLP classifier based Neural Network.

1. **Layer 1 Inputs:** price, EPS, P/E, expected growth, and AAA corporate bond Yield
Layer 1 Outputs: the Intrinsic Value expressed as a dollar amount
2. **Layer 2 Inputs:** P/E, Intrinsic Value, price
Layer 2 Outputs: a binary value (either 1 or 0) and a probabilities metric
During training: The Neural Network (Logistic Regressive) trained on Historical data, comparing P/E, Intrinsic Values, and the current price against the real outcomes of price movement between 2021–2025.
 - Label 1 = Winner: the stock grew over the past 4 years
 - Label 0 = Loser: the stock fell over the past 4 years
 - Possible Action: the NN learns to "veto" Intrinsic Value when it sees a historical pattern of "Value Traps" in relation to the P/E ratios.

The model during production consists of a package of artifacts which contain the model, and the model's StandardScaler. For the most effective forecast, the scaler must be used before feeding the model any new asset metrics, which ensures the model makes predictions on values scaled similarly to its training data.

Key Model Dependencies

1. Joblib - for loading the serialized model artifact file
2. Pandas - when handling data matrices and array transformations
3. Scikit-learn (maybe) - loading MLPClassifier and StandardScaler

Model Inputs

The model needs a Pandas DataFrame row containing [P/E ratio, Intrinsic Value, Price] all formatted as decimal values (floats).

1. P/E ratio - provide the current P/E (accepts \$0 values or negative values)
2. Intrinsic Value - calculate using Benjamin Graham's formula before feeding the model
3. Price - input the current price or end-of-day trailing share price

Crucial Operational Note: when implementing the model any raw inputs must pass through the loaded StandardScaler first via transform() before being handed to the Neural Network.

Model Outputs

The *predict_proba(x_scaled)* method returns the raw probability distribution for both classes (1 and 0) which could be useful for showing model confidence. Meanwhile, the *predict(x_scaled)* function outputs a binary integer value where 1 represents a stock predicted to rise in price, and 0 represents a stock predicted to fall in price.

Training Dataset Sources

Kaggle was the source of all publicly available datasets used during development, while the yFinance API (free tier) provided historical and current stock metrics, from 2017 to 2021. The yFinance API data was used for calculating compounded annual growth rates and labeling stock growths as “overvalued” or “undervalued”. The kaggle datasets were used to extract metrics such as names, tickers, prices, P/E, EPS values, and etc. Below are the links to all Kaggle datasets used during data acquisition:

[US stock index comprehensive data](#)

[s&p 500 EPS and PE ratio](#)

[Current Corporate AAA bond yield value](#)

[s&p 500 growth rate price/returns](#)

Structure of Training Data

The data was housed within a local Microsoft SQL Express Database. Interfacing with the database was done through several T-SQL scripts, written/executed through SSMS, and python 3 scripts written/executed within a virtual environment using SQLAlchemy. Below is a snippet of the SQL script used to create the tables to house the training data.

```

18 CREATE TABLE dbo.stocks (
19     ticker      VARCHAR(10)  PRIMARY KEY,
20     stock_name  VARCHAR(255) NOT NULL,
21     sector      VARCHAR(255) NOT NULL,
22     price       DECIMAL(18,2) NOT NULL,
23     earnings    DECIMAL(18,2) NOT NULL,
24     pe_ratio    DECIMAL(18,2) NOT NULL,
25     market_cap  DECIMAL(20,2) NOT NULL
26 );
27
28 CREATE TABLE dbo.growth_rates (
29     ticker      VARCHAR(10),
30     start_value  DECIMAL(18, 2) NOT NULL,
31     end_value    DECIMAL(18, 2) NOT NULL,
32     growth_rate  DECIMAL(18, 2) NOT NULL,
33     corp_bond_yield DECIMAL(18, 2) NOT NULL
34     CONSTRAINT fk_stock_growth_rates
35     FOREIGN KEY (ticker)
36     REFERENCES dbo.stocks(ticker)
37 );
38
39 CREATE TABLE dbo.historic_valuations (
40     ticker      VARCHAR(10),
41     valuation    VARCHAR(12),
42     CONSTRAINT  fk_historic_valuations
43     FOREIGN KEY (ticker)
44     REFERENCES  dbo.stocks(ticker)
45 );
46

```

Figure 2: Structure T-SQL used to create the tables for model training data

During model training, pulling data into a Jupyter Notebook was done through a combination of the SQL query shown below and a Pandas dataframe.

```

query = """
SELECT
    s.ticker, s.price, s.earnings, s.pe_ratio,
    g.growth_rate, g.corp_bond_yield, h.valuation
FROM dbo.stocks s
JOIN dbo.growth_rates g ON s.ticker = g.ticker
JOIN dbo.historic_valuations h ON s.ticker = h.ticker
WHERE s.price > 0
"""

```

Figure 3: Query string from the model training code

Evaluating The Model

The model development was completed by May 2026 along with the testing Jupyter Notebook script, and after 4 months the evaluation was launched on September 04, 2026.

The scenario used for model evaluation was assuming that 100 shares of each ticker from the testing dataset was considered theoretically for purchase in an investment account. If the model predicted (1) for a ticker then 100 shares were bought, alternatively if it predicted (0) no action was taken. Then the current prices of the tickers were compared with the prices on the testing dataset, and the theoretically purchased shares were checked to see how much value was lost or gained.

```

=== PORTFOLIO CATEGORY BREAKDOWN ===

```

	category	count	total_pnl	avg_pct_change
Avoided Loss (Skipped)		15	-29718.30	-14.650845
Loss-Making Buy		40	-286740.50	-13.150378
Missed Gain (Skipped)		24	55251.64	1036.035474
Profitable Buy		63	246069.00	17.052745

Figure 4: Model evaluation breakdown with theoretical scenario

Key Operational Metrics:

- Win Rate on Executed Trades: 61.17% (63 winning buys vs. 40 losing buys)
- Overall Signal Decision Accuracy: 54.93% (78 correct choices out of 142 total opportunities)
- Skip Signal Precision: 38.46% (15 avoided losses vs. 24 missed gains)

What Worked Well:

- A 61.2% win rate on buy selections is above a 50/50 random chance and demonstrates a somewhat accurate stock-picking capability which isn't just noise.
- The model generated +\$246,069.00 across 63 profitable positions, proving it successfully identifies stocks with strong upside momentum.

Where the Model Failed:

- The model was significantly worse at skipping trades than selecting buys. Out of 39 skipped stocks, 61.5% (24 stocks) turned out to be Missed Gains, including massive gainers averaging +1,036% growth
- A single stock, KLAC (-\$156,764.00), accounted for 54.7% of all capital lost, and without it the executed trade portfolio would have been +\$116,092.50 in net profit.

Final Assessment

The model's underlying signal engine works for identifying long opportunities. However, as an end-to-end trading system, it fails due to poor risk management infrastructure. Since investment like KLAC can be avoided by implementing a strict stop-loss system.